

Reviewer Pack — Invariant Degree Gap in Read-Twice Branching Programs for IP...

Entry 5d978c228eaa · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 15:06:31 UTC

Invariant Degree Gap in Read-Twice Branching Programs for IP_2

Entry ID: 5d978c228eaa **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 15:06:31 UTC

1. Conjecture

Field A (mathematical branch): INVARIANT_THEORY **Field B** (complexity object): BRANCHING_PROGRAMS

Statement:

For a read-twice branching program P computing a function $f: \{0,1\}^n \rightarrow \{0,1\}$, let $\deg_{\text{inv}}(f)$ denote the minimal degree of a non-trivial polynomial invariant under the action of the general linear group $GL(n,2)$ that vanishes on f . Then $\deg_{\text{inv}}(f) = O(\log \text{size}(P))$ for all f computable by read-twice BPs, but $\deg_{\text{inv}}(\text{IP}_2) = \Omega(n)$ where IP_2 is the inner product mod 2 function.

Rationale (proposer's reasoning):

Invariant theory provides polynomial functions invariant under group actions, which can distinguish functions with different symmetry properties. Read-twice BPs for IP_2 must encode high-degree invariants due to the function's non-linearity, while general read-twice functions have low-degree invariants. This conjecture bridges algebraic geometry and circuit complexity.

Taxonomy category: FOURIER_ANALYTIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): aaf9eef87022620c

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def generate_random_read_twice_bp(n: int) -> list:
    bp = []
    for _ in range(2**n):
        row = [random.choice([0, 1]) for _ in range(n)]
        bp.append(row)
    return bp

def inner_product_mod_2(x: list, y: list) -> int:
    return sum(xi * yi for xi, yi in zip(x, y)) % 2

def polynomial_degree(poly: dict) -> int:
    return max(degree for degree, _ in poly.items())

def is_invariant(poly: dict, bp: list) -> bool:
    n = len(bp[0])
    for i in range(n):
        new_bp = []
        for row in bp:
            new_row = [row[j] if j != i else (1 - row[j]) for j in range(n)]
            new_bp.append(new_row)
        new_value = inner_product_mod_2([new_bp[row][i] for row in range(len(bp))], [poly[(i,)]])
        if new_value != poly[(i,)]:
            return False
    return True

def find_min_invariant_degree(bp: list, ip2_value: int) -> int:
    n = len(bp[0])
    degree = 1
    while True:
        poly = {}
        for i in range(n):
            poly[(i,)] = random.choice([0, 1])
        if is_invariant(poly, bp) and polynomial_degree(poly) > 0:
            return degree
```

```

        degree += 1

def run_trial(seed: int) -> dict:
    n_values = [5, 10, 15, 20, 30, 40]
    results = []
    for n in n_values:
        random.seed(seed)
        bp = generate_random_read_twice_bp(n)
        ip2_value = inner_product_mod_2([random.choice([0, 1]) for _ in range(n)], [random.choice([0, 1]) for _ in range(n)])
        min_invariant_degree = find_min_invariant_degree(bp, ip2_value)
        results.append({
            "n": n,
            "min_invariant_degree": min_invariant_degree
        })
    metric_name = "Min Invariant Degree"
    metric_value = sum(result["min_invariant_degree"] for result in results) / len(results)
    instances_tested = len(results)
    conjecture_holds = all(result["min_invariant_degree"] <= math.log(2**n, 2) for result in results)
    counterexample = "" if conjecture_holds else "mapping_undefined"
    return {
        "metric_name": metric_name,
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(seed) for seed in sys.argv[1:]] or [2**i - 1 for i in range(5, 35)]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
    std_metric_value = math.sqrt(sum((result["metric_value"] - mean_metric_value)**2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

```

Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_573776bd.py", line 84, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_573776bd.py", line 61, in run_trial
    min_invariant_degree = find_min_invariant_degree(bp, ip2_value)
                          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_573776bd.py", line 50, in find_min_inva
    if is_invariant(poly, bp) and polynomial_degree(poly) > 0:
                                   ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
TypeError: '>' not supported between instances of 'tuple' and 'int'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to type error in degree comparison; unable to evaluate conjecture's validity | next: Fix polynomial_degree return type to integer and revalidate invariant degree calculations

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	72881
2	preregistration	ollama_remote	qwen3:8b	0	0	24171
3	novelty	ollama_remote	qwen3:8b	0	0	20941
4	novelty	ollama_remote	qwen3:8b	0	0	14662
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15888
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10519
7	judge	ollama_remote	qwen3:8b	0	0	17784

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 176847 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/5d978c228eaa.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/5d978c228eaa.jsonl` for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/5d978c228eaa.tar.gz` (if generated)